Experiment 9

Clustering: K-Means Algorithm

Aim: To perform clustering using the K-Means algorithm and determine the optimal number of clusters using the Elbow method.

Theory

1. Labeled vs Unlabeled Data

Labeled Data:
Labeled data refers to a dataset where each input sample is associated with a known output label or target value. These labels guide the learning process in supervised learning algorithms (e.g., classification — assign one of several classes, or regression — predict a continuous value).

For example, in an email spam detection system, each email in the dataset may be labeled as "Spam" or "Not Spam." The algorithm learns patterns from these labeled examples to classify new emails.

Unlabeled Data:
Unlabeled data refers to a dataset where only input features are available and no output labels or predefined categories are provided. Since there are no target values, the algorithm must automatically identify patterns, structures, or relationships present in the data. This type of data is commonly used in unsupervised learning.

A common technique used to analyze unlabeled data is clustering, where the algorithm groups similar data points into clusters based on their characteristics.

For example, a dataset contains age and income of customers, but no information about customer categories. A clustering algorithm can group them into clusters such as high-income, medium-income, and low-income customers based on similarity.

2. Introduction to Clustering

Clustering is the process of grouping similar data points into clusters so that points within a cluster are more similar to each other than to those in other clusters. Clustering is commonly used to explore a dataset to either identify the underlying patterns in it or to create a group of characteristics.

Clustering has many practical applications, including:

- Customer segmentation in marketing
- Document grouping in text mining
- Image segmentation in computer vision
- Anomaly detection in fraud detection systems

3. Introduction to K-Means Clustering

K-Means is one of the most widely used unsupervised clustering algorithms. It belongs to the category of partition-based clustering algorithms. The main objective of K-Means is to divide a dataset into K distinct clusters, where each cluster is represented by a central point called the centroid.

The algorithm works by assigning each data point to the cluster whose centroid is closest to it, usually measured using Euclidean distance. As the algorithm progresses, the positions of centroids are updated so that they represent the mean position of all data points within the cluster.

4. Geometrical Interpretation of K-Means

From a geometric perspective, each data point in a dataset can be represented as a point in a feature space. For example, if the dataset has two features such as height and weight, each observation can be plotted as a point in a two-dimensional coordinate plane.

In K-Means clustering, each cluster is represented by a centroid, which acts as the center of the cluster. The algorithm measures the distance between each data point and the centroids (using Euclidean distance) and assigns the point to the nearest centroid.

As a result, the entire feature space is partitioned into regions, where each region corresponds to a cluster. Every data point within a region belongs to the cluster whose centroid is closest to it. During the iterative process, centroids move to the mean position of the points assigned to them, gradually forming well-defined clusters.

5. Mathematical Representation

In K-Means clustering, the similarity between data points and cluster centroids is measured using a the Euclidean distance. The Euclidean distance between two points $\mathrm{A}(\mathrm{X}_1, \mathrm{Y}_1)$and $\mathrm{B}(\mathrm{X}_2, \mathrm{Y}_2)$is given by:

$$\mathrm{d}(\mathrm{A}, \mathrm{B}) = \sqrt{(\mathrm{X}_2 - \mathrm{X}_1)^2 + (\mathrm{Y}_2 - \mathrm{Y}_1)^2}$$

This distance measure helps determine which centroid a data point is closest to, and accordingly the point is assigned to that cluster.

The main objective of the K-Means algorithm is to minimize the total within-cluster variance, which indicates how far the data points in a cluster are from their corresponding centroid. A lower variance means that the data points are closer to the centroid, resulting in more compact clusters.

The algorithm minimizes the sum of squared distances between each data point and its cluster centroid. This objective function is expressed as:

$$J(K) = \sum_{k=1}^{K} \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

Where:

- $J(K)$= Cost function for Kclusters
- C_k= Cluster k
- x_i= Data point
- μ_k= Centroid (mean) of cluster k

6. K-Means Clustering Algorithm

- **Step 1:** Choose K (number of clusters)
- **Step 2:** Initialize K centroids:

    - **Random:** Pick K random data points as initial centroids
    - **Pick first centroid randomly:** For each subsequent centroid, pick point with probability proportional to squared distance from nearest existing centroid
- **Step 3: REPEAT** until convergence:
    - **Step 3a:** Assignment Step
        - For each data point:
            - Calculate distance to all K centroids
            - Distance formula: $d = \sqrt{[(x_1-c_1)^2 + (x_2-c_2)^2 + ...]}$
            - Assign point to cluster with nearest centroid
    - **Step 3b:** Update Step
        - For each cluster k:
            - Calculate new centroid as mean of all points in cluster
            - New centroid coordinates = $(\Sigma x_i/n, \Sigma y_i/n, ...)$
    - **Step 3c:** Check Convergence
        - If centroids haven't moved (or moved less than threshold) then **STOP**
        - If maximum iterations reached then **STOP**
        - Otherwise, repeat from Step 3a
- **Step 4:** Output final cluster assignments and centroid positions

7. Choosing the Number of Clusters: Elbow Technique

One of the key challenges in K-Means clustering is selecting the appropriate number of clusters (K). Choosing too few clusters may group dissimilar data points together, while too many clusters may lead to unnecessary fragmentation of the data.

A commonly used method to determine a suitable value of K is the Elbow Method. In this technique, the K-Means algorithm is run multiple times using different values of K, and for each value the Within-Cluster Sum of Squares (WCSS) is calculated.

The WCSS measures how closely the data points in a cluster are grouped around their centroid. By comparing WCSS values for different numbers of clusters, it becomes possible to identify an appropriate number of clusters for the dataset.

8. Mathematical Relation in Elbow Method

The Within-Cluster Sum of Squares (WCSS) is a measure used to evaluate the compactness of clusters in K-Means clustering. It represents the sum of squared distances between each data point and the centroid of the cluster to which it belongs. A smaller WCSS value indicates that the data points are closer to their cluster centroid, meaning the clusters are more compact.

Mathematically, WCSS is defined as:

$$WCSS = \sum_{k=1}^{K} \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

Where:

- C_k= cluster k
- x_i= data point
- μ_k= centroid of cluster k

As the number of clusters (K) increases, the WCSS value generally decreases because data points are divided into smaller groups and are closer to their respective centroids.

However, after a certain value of K, the decrease in WCSS becomes much smaller. The objective is to find the point where the reduction in WCSS starts to slow down significantly, indicating that adding more clusters does not substantially improve the clustering quality. This point is used to determine the optimal number of clusters.

9. Geometrical Interpretation of the Elbow Method

The elbow method involves plotting:

- Number of clusters (K) on the x-axis
- WCSS on the y-axis

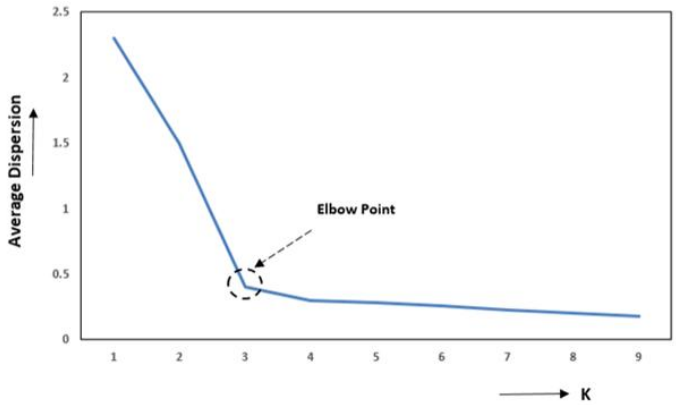


Figure 1: Elbow Method

As shown in figure 1, the WCSS decreases rapidly as more clusters are added. However, after a certain value of K, the rate of decrease slows down, and the graph begins to flatten. The point where the curve forms a sharp bend or "elbow" represents the optimal number of clusters. This indicates that adding more clusters beyond this point provides only marginal improvement in clustering performance.

10. Merits of K-Means Clustering

- K-Means is simple to understand and easy to implement.
- It is computationally efficient, making it suitable for large datasets.

- The algorithm produces clear cluster representations using centroids, which are easy to interpret.

11. Demerits of K-Means Clustering

- The algorithm is sensitive to the initial selection of centroids. Different initial centroid positions may lead to different clustering results, and sometimes the algorithm may converge to a local optimum instead of the best solution.
- K-Means assumes that clusters are spherical and evenly sized. Therefore, it performs poorly when the dataset contains non-spherical clusters or clusters with different shapes and densities.
- The algorithm requires the number of clusters (K) to be specified in advance, which may not always be known beforehand and can affect the quality of clustering if chosen incorrectly.

K-Means Clustering - Practical Implementation

Step 1: Importing Libraries

Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.metrics import davies_bouldin_score
print("Libraries Imported")
```

Output:

```
Libraries Imported
```

Step 2: Loading Dataset

Reading Mall_Customers dataset

```
data = pd.read_csv("Mall_Customers.csv")
print("Dataset loaded successfully")
```

Output:

```
Dataset loaded successfully
```

Step 3: Data Analysis

Display the first 5 rows of the dataset

```
data.head()
```

Output:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Display the last 5 rows of the dataset

```
data.tail()
```

Output:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

Display the summary statistics of the dataset

```
data.describe()
```

Output:

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

Display the information of the dataset

```
data.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
```

Output:

#	Column	Non-Null Count	Dtype
0	CustomerID	200 non-null	int64
1	Gender	200 non-null	object
2	Age	200 non-null	int64
3	Annual Income (k$)	200 non-null	int64
4	Spending Score (1-100)	200 non-null	int64

Output:

```
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

Check for missing values

```
data.isnull().sum()
```

Output:

Column	Missing Values
CustomerID	0
Gender	0
Age	0
Annual Income (k$)	0
Spending Score (1-100)	0

Output:
```
dtype: int64
```

 # Display the shape of the dataset

```
data.shape
```

Output:
```
(200, 5)
```

Step 4: Data Preprocessing

 # Convert the 'Gender' column from categorical values (Male/Female) to numeric labels (0/1) and preview the updated

```
data['Gender'] = data['Gender'].map({'Male': 0, 'Female': 1})
data.head()
```

Output:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	0	19	15	39
1	2	0	21	15	81
2	3	1	20	16	6
3	4	1	23	16	77
4	5	1	31	17	40

 # Select Annual Income and Spending Score as feature variables for clustering and display a confirmation message

```
X = data[['Annual Income (k$)', 'Spending Score (1-100)']]
print("Features selected for clustering: Annual Income and Spending Score")
```

Output:
```
Features selected for clustering: Annual Income and Spending Score
```

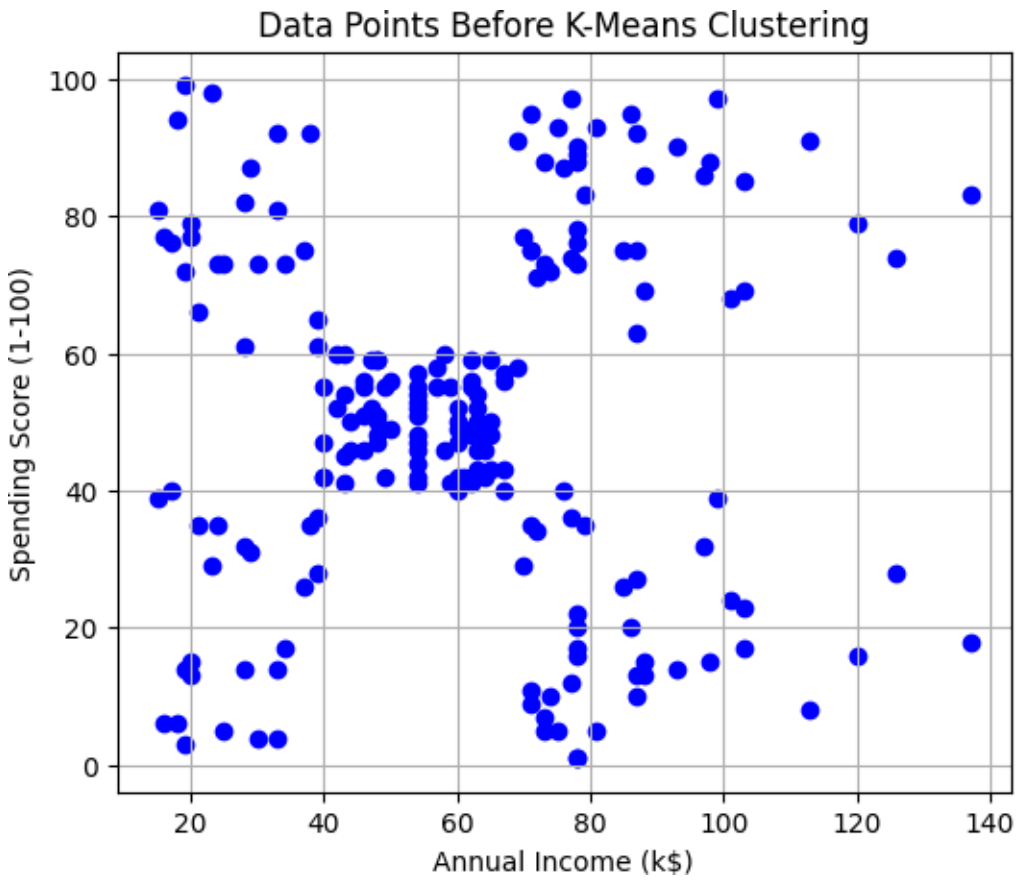
 # Visualize the original data points by plotting Annual Income against Spending Score before applying K-Means clust

```
plt.figure(figsize=(6,5))
plt.scatter(
    X['Annual Income (k$)'],
    X['Spending Score (1-100)'],
    color='blue'
)

plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Data Points Before K-Means Clustering")
plt.grid()
plt.show()
```

Output:

```
Data Points Before K-Means Clustering
```



Step 5: Model Training

 # Apply StandardScaler to normalize the selected features so they have zero mean and unit variance

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Feature scaling complete using StandardScaler")
```

Output:

```
Feature scaling complete using StandardScaler
```

 # Compute WCSS for different numbers of clusters and plot the Elbow Curve to identify the optimal value of K

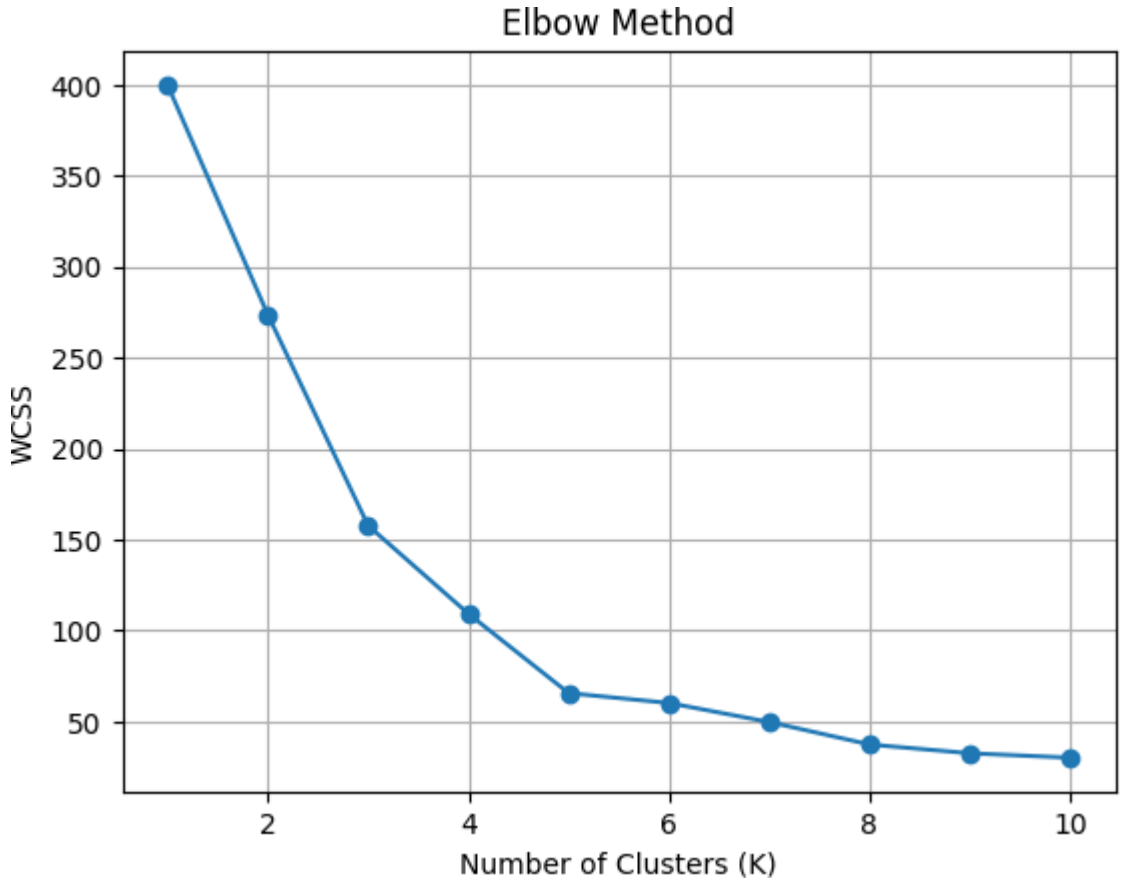
```
wcss = []

for k in range(1,11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.plot(range(1,11), wcss, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.grid()
plt.show()
```

Output:

```
Elbow Method (k=5)
```



 # Initialize the K-Means algorithm with 5 clusters and train it on the scaled feature data

```
kmeans = KMeans(n_clusters=5, random_state=42)
kmeans.fit(X_scaled)
print("Model trained successfully")
```

Output:

```
Model trained successfully
```

KMeans

```
KMeans(n_clusters=5, random_state=42)
```

Step 6: Model Evaluation

 # Assign cluster labels to the original dataset and display the first 5 rows with the new 'Cluster' column

```
labels = kmeans.labels_
data['Cluster'] = labels
data[['Annual Income (k$)', 'Spending Score (1-100)', 'Cluster']].head()
```

Output:

	Annual Income (k$)	Spending Score (1-100)	Cluster
0	15	39	4
1	15	81	2
2	16	6	4
3	16	77	2
4	17	40	4

 # Visualize the clusters by plotting Annual Income against Spending Score, with colors representing cluster labels an

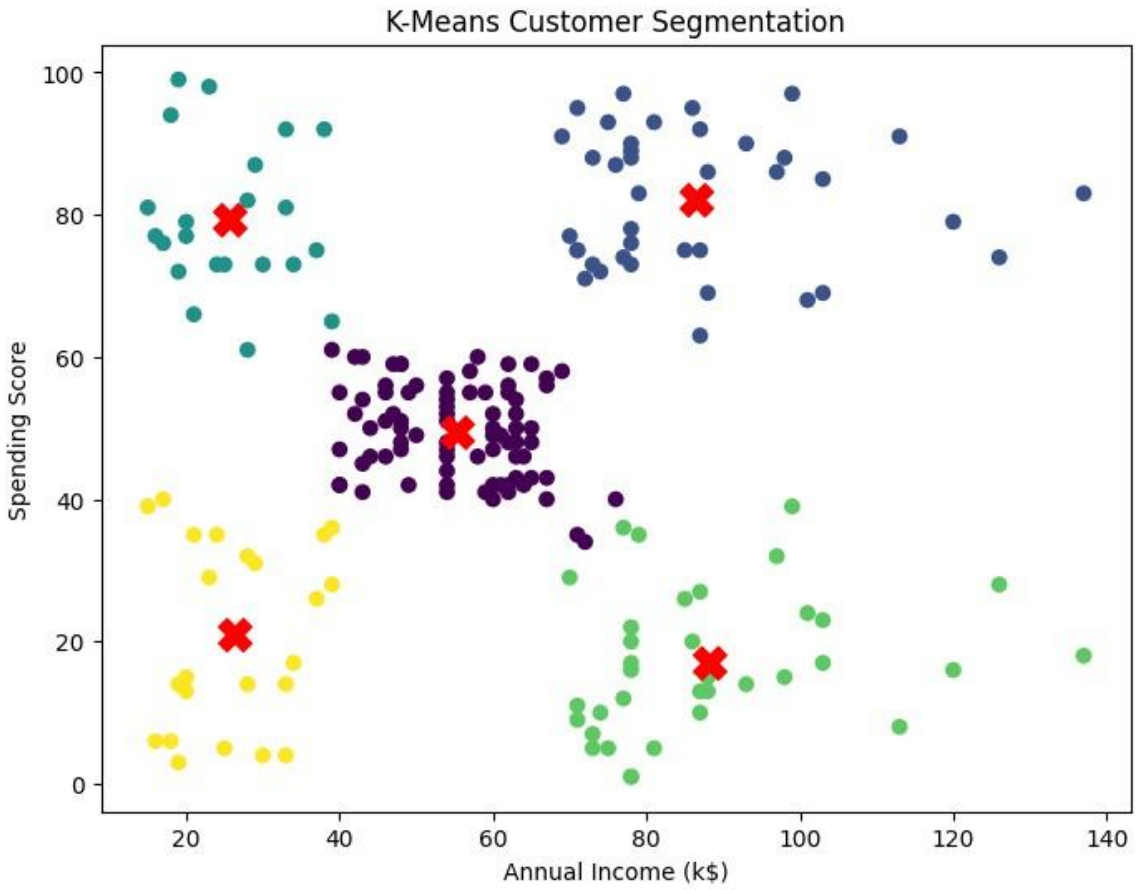
```
plt.figure(figsize=(8,6))
plt.scatter(
    X['Annual Income (k$)'],
    X['Spending Score (1-100)'],
    c=labels,
    cmap='viridis'
)

centers = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(
    centers[:,0],
    centers[:,1],
    c='red',
    s=200,
    marker='X'
)

plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score")
plt.title("K-Means Customer Segmentation")
plt.show()
```

Output:

```
K-Means Customer Segmentation
```



 # Calculate and display model evaluation metrics: Silhouette Score, WCSS (Inertia), and Davies-Bouldin Index

```
score = silhouette_score(X_scaled, labels)
print("Silhouette Score:", score)

wcss = kmeans.inertia_
print("WCSS (Inertia):", wcss)

db_score = davies_bouldin_score(X_scaled, labels)
print("Davies-Bouldin Index:", db_score)
```

Output:

```
Silhouette Score: 0.5546571631111091
WCSS (Inertia): 65.56840815571681
Davies-Bouldin Index: 0.5722356162263352
```

Step 7: Model Simulation

 # Interactive K-Means Simulation

```
Click the 'Enter Interactive Animation' button to open the full-screen simulator.
Explore how K-Means Clustering segments the customer dataset in real-time.
```

Output:

```
Full-Screen Simulator Ready. Click the 'Enter Interactive Animation' button to start exploring.
```